DevOps Lab: Theory and Viva Q&A

Nov 10, 2026 · @Sam

This document covers the theory behind the five DevOps lab experiments (Git, Maven, Docker, Kubernetes, Jenkins with Slack) and likely viva questions with short answers.

Experiment 1: Git (Source Code Management)

Theory

Version control records every change to a project so you can go back to any earlier state, see who changed what, and work in parallel with others. **Git** is a distributed version control system: every developer has a full copy of the repository and its history, so most work happens offline.

The three areas. A file moves through three places:

1. **Working directory:** the files you edit.
2. **Staging area (index):** changes you have marked with `git add` for the next commit.
3. **Repository (`.git`):** committed snapshots, saved with `git commit`.

Core commands

Command	Purpose
`git init`	Create a new repository in the current folder
`git add <file>`	Move changes to the staging area
`git commit -m "msg"`	Save the staged changes as a snapshot
`git log`	Show commit history
`git status`	Show changed, staged and untracked files
`git branch <name>`	Create a branch
`git checkout <name>`	Switch branch
`git merge <name>`	Merge the named branch into the current one
`git config --global --list`	Show user name, email and other settings

Branching. A branch is a movable pointer to a commit. Branches let you build a feature or fix without touching `master`, then merge it back. In the lab, `b2` was created from `master`.

Merge conflict. Git merges automatically when two branches change different parts of a file. If both changed the **same lines**, Git cannot choose and stops with `CONFLICT (content)`. It writes markers into the file:

```
<<<<<<< HEAD
(your current branch's version)
=======
(the incoming branch's version)
>>>>>>> b2
```

To resolve it: edit the file to the final text, delete the three marker lines, run `git add`, then `git commit`.

Viva questions

1. What is the difference between Git and GitHub? Git is the version control tool that runs on your machine. GitHub is a hosting service for Git repositories with sharing and collaboration features.

2. What does `git init` do? It creates a hidden `.git` folder that turns the current folder into a repository.

3. What is the staging area and why does it exist? It is a holding area between your files and the commit. It lets you choose exactly which changes go into a commit.

4. Difference between `git add` and `git commit`? `git add` stages changes. `git commit` permanently records the staged changes in history with a message.

5. What does `git log` show? The commit history: commit hash, author, date and message, newest first.

6. What is a branch? Why use one? A lightweight pointer to a commit. It isolates work on a feature or fix from the stable code.

7. What is a merge conflict and when does it happen? It happens when two branches change the same part of the same file (or one deletes a file the other edits), so Git cannot merge automatically.

8. How did you create the conflict in the lab? I edited the same `print` line in `add.py` differently on `master` and `b2`, then ran `git merge b2`.

9. What do `<<<<<<< HEAD`, `=======` and `>>>>>>> b2` mean? `HEAD` is the current branch's version, `b2` is the incoming branch's version, and `=======` separates them.

10. How do you resolve a merge conflict? Edit the file to the correct final content, remove the markers, `git add` the file, then `git commit`. To cancel the merge, use `git merge --abort`.

11. What is the difference between `git merge` and `git rebase`? Merge joins two histories with a merge commit and keeps history as it happened. Rebase replays your commits on top of another branch for a straight history, but rewrites commits.

12. What is the difference between `git pull` and `git fetch`? `fetch` downloads remote changes without changing your files. `pull` is `fetch` plus a merge.

13. What is HEAD? A pointer to the commit (usually the tip of the current branch) that your working directory is based on.

14. Why did you run `git config --global --list`? To check the user name and email that Git records as the author of commits.

15. How do you undo the last commit but keep the changes? `git reset --soft HEAD~1`.

Experiment 2: Apache Maven (Build and Dependency Management)

Theory

Maven is a build automation and dependency management tool for Java. Instead of listing build steps by hand, you describe the project in a `pom.xml` and Maven follows conventions to compile, test and package it.

POM (Project Object Model). `pom.xml` is the project's configuration file. Key elements:

Element	Meaning
`groupId`	Organization or package namespace, such as `Test`
`artifactId`	Name of the project or JAR
`version`	Version of the artifact, such as `1.0-SNAPSHOT`
`packaging`	Output type: `jar`, `war`, `pom`

Element	Meaning
`dependencies`	Libraries the project needs
`properties`	Settings such as the Java version

The three together (`groupId:artifactId:version`) are the **coordinates** that uniquely identify an artifact.

Standard directory layout. Maven expects a fixed structure, which is why little configuration is needed:

```
project/
  pom.xml
  src/main/java         application code
  src/main/resources    config files
  src/test/java         unit tests
  target/               build output (JAR, classes)
```

Build lifecycle. A lifecycle is an ordered set of phases. Running a phase runs every phase before it. The default lifecycle's main phases:

1. `validate` : check the project is correct.
2. `compile` : compile source code into `target/classes` .
3. `test` : run unit tests (JUnit).
4. `package` : bundle the compiled code into a JAR or WAR.
5. `verify` : run checks on the package.
6. `install` : copy the artifact into the local repository (`~/.m2`).
7. `deploy` : copy it to a remote repository.

Other lifecycles are `clean` (delete `target`) and `site` (generate documentation).

Dependency management. You declare a dependency (here, `json-simple`) and Maven downloads it and the libraries it needs (**transitive dependencies**). Libraries come from repositories: the **local** repository (`~/.m2/repository`), **central** (Maven Central) and any **remote** or company repository. A **scope** limits where a dependency is used: `compile` (default), `test` , `provided` , `runtime` .

What the lab did. A Maven project was created in Eclipse, `json-simple` was added to `pom.xml` , and `DarSON.java` used `JSONParser` to read `student.json` and print the first and last name.

Viva questions

1. What is Maven and why is it used? A build and dependency management tool. It automates compiling, testing and packaging, and downloads libraries automatically.

2. What is `pom.xml`? The Project Object Model file. It holds project coordinates, dependencies, plugins and build settings.

3. What are Maven coordinates? `groupId`, `artifactId` and `version`. They uniquely identify an artifact.

4. Name the main phases of the default lifecycle. `validate`, `compile`, `test`, `package`, `verify`, `install`, `deploy`.

5. What happens when you run `mvn package`? Maven runs all earlier phases (validate, compile, test) and then builds the JAR into `target/`.

6. Difference between `mvn install` and `mvn package`? `package` creates the JAR in `target/`. `install` also copies it into the local repository so other local projects can use it.

7. What does `mvn clean` do? Deletes the `target` directory, removing old build output.

8. What is a Maven repository? Name the types. A store of artifacts and dependencies. Local (`~/.m2`), central (Maven Central) and remote (company or third-party).

9. What is a transitive dependency? A library that your dependency itself depends on. Maven downloads it for you.

10. What are dependency scopes? They control where a dependency is available: `compile`, `test`, `provided`, `runtime`. JUnit is usually `test` scope.

11. Which dependency did you add and why? `json-simple`, to parse the `student.json` file using `JSONParser` and `JSONObject`.

12. What is the difference between a JAR and a WAR? A JAR packages a Java library or application. A WAR packages a web application to run on a server such as Tomcat.

13. Where do unit tests go and which framework runs them? In `src/test/java`. JUnit is the usual framework, run by the Surefire plugin during the `test` phase.

14. Maven vs Ant or Gradle? Ant needs every build step written out. Maven uses conventions and a declarative POM. Gradle uses a Groovy or Kotlin script and is often faster.

15. What is a SNAPSHOT version? A version still in development (`1.0-SNAPSHOT`). Maven may re-download it, unlike a fixed release version.

Experiment 3: Docker (Containerization)

Theory

Containerization packages an application with everything it needs (code, runtime, libraries, config) so it runs the same on any machine. **Docker** is the most widely used platform for building and running containers.

Container vs virtual machine. A VM includes a full guest operating system on top of a hypervisor, so it is heavy and slow to start. A container shares the host OS kernel and isolates only the application, so it is small and starts in seconds.

Key concepts

Concept	Meaning
Image	Read-only template made of layers; used to create containers
Container	A running instance of an image
Dockerfile	Text file of instructions to build an image
Registry	Storage for images, such as Docker Hub
Repository	A named collection of image versions (tags) in a registry
Tag	Label for an image version, such as `latest`
Docker Engine	The daemon that builds and runs containers

Dockerfile used in the lab

```
FROM httpd:latest
COPY index.html /usr/local/apache2/htdocs/
```

`FROM` sets the base image. `COPY` puts the HTML file into the folder Apache serves. Other common instructions: `RUN` (run a command while building), `WORKDIR`, `EXPOSE` (document a port), `CMD` and `ENTRYPOINT` (the command that starts the container). Each instruction

creates a **layer**, and unchanged layers are cached to speed up rebuilds.

Commands used

Command	Purpose
`docker build -t samdev .`	Build an image from the Dockerfile in the current folder and name it
`docker run -d --name c1 -p 9091:80 samdev`	Start a container in the background, mapping host port 9091 to container port 80
`docker ps`	List running containers
`docker tag samdev sammdocker26/samdev`	Give the image a registry name (`user/repo`)
`docker push sammdocker26/samdev`	Upload the image to Docker Hub
`docker pull legendsat18/sathvik`	Download an image from Docker Hub

Port mapping. `-p 9091:80` means requests to port 9091 on your computer are forwarded to port 80 inside the container, where Apache listens.

Apache HTTP Server vs Tomcat. `httpd` serves static files such as HTML. Tomcat is a Java servlet container for Java web applications (WAR files) and also serves static files, on port 8080 by default. The lab brief names Tomcat; with it the Dockerfile would be `FROM tomcat` and the files would be copied to `/usr/local/tomcat/webapps/ROOT/` .

Viva questions

1. What is Docker? A platform that packages applications and their dependencies into containers so they run the same everywhere.

2. Difference between a container and a virtual machine? A VM has its own guest OS and is heavy. A container shares the host kernel and is lightweight and fast to start.

3. Difference between an image and a container? An image is a read-only template. A container is a running instance created from it.

4. What is a Dockerfile? A text file with the instructions Docker follows to build an image.

5. Explain `FROM` and `COPY` in your Dockerfile. `FROM httpd:latest` selects the Apache base image. `COPY index.html /usr/local/apache2/htdocs/` adds the web page to Apache's document root.

6. What does `docker build -t samdev .` mean? Build an image from the Dockerfile in the current folder (`.`) and name it `samdev` (`-t` sets the tag).

7. What does `-d` mean in `docker run`? Detached mode: the container runs in the background.

8. Explain `-p 9091:80`. It maps host port 9091 to container port 80, so the site opens at `localhost:9091`.

9. What is Docker Hub? A public registry where images are stored and shared. You push your own and pull others.

10. Why did you run `docker tag` before `docker push`? Docker Hub needs the image name in the form `username/repository`. Tagging gives the local image that name.

11. What is the difference between `CMD` and `ENTRYPOINT`? Both define what runs at start. `CMD` provides default arguments that are easy to override. `ENTRYPOINT` sets the main executable.

12. What is the difference between `RUN` and `CMD`? `RUN` executes during the build and saves the result in a layer. `CMD` runs when the container starts.

13. What are image layers? Each Dockerfile instruction adds a read-only layer. Layers are cached and shared between images.

14. Why did the lab image use `httpd` and not Tomcat? The page is static HTML, which Apache HTTP Server serves directly. Tomcat is needed for Java web apps and could be used by changing the base image.

15. How do you stop and remove a container? `docker stop c1` then `docker rm c1`.

16. How do you see a container's logs or open a shell in it? `docker logs c1` and `docker exec -it c1 bash`.

Experiment 4: Kubernetes (Container Orchestration)

Theory

Kubernetes (K8s) is an open-source platform that runs and manages containers across a cluster of machines. A single container is easy to run with Docker; Kubernetes handles what comes next: starting many copies, restarting failed ones, spreading traffic, scaling and rolling out updates.

Architecture

Part	Role
Control plane	Makes decisions for the cluster
API server	Front door; `kubectl` talks to it
etcd	Key-value store holding the cluster state
Scheduler	Chooses a node for each new pod
Controller manager	Keeps the actual state equal to the desired state
Node (worker)	Machine that runs pods
kubelet	Agent on each node that starts and watches containers
kube-proxy	Handles networking rules for Services

Objects used in the lab

Object	Meaning
Pod	Smallest unit; one or more containers sharing an IP and storage
ReplicaSet	Keeps a set number of identical pods running
Deployment	Manages ReplicaSets; gives scaling and rolling updates
Service	Stable IP and name that load-balances across matching pods
Label and selector	Key-value tags on pods, and the rule a Service uses to find them

Deployment manifest (`deployment.yaml`). Declares the desired state: the name `sams-deployment`, the number of `replicas`, a `selector` matching label `app: sams`, and a pod template with the container image and port 80.

Service manifest (`service.yaml`). Creates the Service `kuber11` with a `selector` of `app: sams`, so it sends traffic to every pod carrying that label. Service types:

1. **ClusterIP** (default): reachable only inside the cluster.

2. **NodePort:** also opens a port (30000-32767) on every node. Used in the lab: `80:30092/TCP`.
3. **LoadBalancer:** asks a cloud provider for an external load balancer.

Scaling. Change the replica count with `kubectl scale deployments sams-deployment --replicas=5`, or edit the manifest and re-apply. Kubernetes adds or removes pods to match.

Self-healing. Kubernetes compares desired and actual state continuously. When a pod was deleted, the ReplicaSet saw one fewer pod than the desired count and started a new one.

Load balancing. The Service keeps an **Endpoints** list of the pod IPs behind it (`10.244.0.10:80` and others) and spreads requests across them.

Declarative approach. You state what you want in YAML and `kubectl apply -f` makes the cluster match. Re-applying a changed file (the lab's "configured") updates the existing object.

Commands used

Command	Purpose
`kubectl apply -f file.yaml`	Create or update objects from a manifest
`kubectl get deployments / pods / svc`	List objects
`kubectl get pods --show-labels`	Show pod labels
`kubectl get endpoints kuber11`	Show the pod IPs behind a Service
`kubectl scale deployments <name> --replicas=N`	Change replica count
`kubectl delete pod <name>`	Delete a pod (it is recreated)
`kubectl port-forward service/kuber11 8081:80`	Forward a local port to the Service
`kubectl version`	Show client and server versions

Viva questions

1. What is Kubernetes and why is it needed? An orchestration platform that automates deployment, scaling, load balancing and recovery of containers across many machines.

2. What is a pod? The smallest deployable unit: one or more containers sharing network and storage.

3. What is a Deployment? An object that manages a set of identical pods through a ReplicaSet. It provides scaling, rolling updates and rollback.

4. What is a ReplicaSet? It makes sure the desired number of pod replicas is always running.

5. What is a Service and why is it required? Pods get new IPs when they restart. A Service gives a stable address and load-balances traffic across the matching pods.

6. How does the Service find the pods? Through labels. The Service selector `app: sams` matches pods labeled `app=sams`.

7. Compare ClusterIP, NodePort and LoadBalancer. ClusterIP is internal only. NodePort exposes a port on each node. LoadBalancer provisions an external cloud load balancer.

8. How did you scale the application? `kubectl scale deployments sams-deployment --replicas=5`, then 2 and 4. The pod count changed to match each time.

9. Why did `kubectl scale deployments sams` fail? The deployment is named `sams-deployment`. The name must match exactly (`NotFound`).

10. What is self-healing? How did you show it? Kubernetes replaces failed pods automatically. I deleted a pod with `kubectl delete pod` and a new one started within seconds.

11. What happened when the first `kubectl apply` failed? The manifest had an unknown field (a typo in `name`), so the API server rejected it. After fixing the YAML it was created.

12. What does "created" vs "configured" mean after `kubectl apply`? "created" means a new object was made. "configured" means an existing object was updated with changes.

13. What are endpoints? The list of pod IP and port pairs that a Service sends traffic to.

14. Why use `kubectl port-forward`? To reach a Service or pod from your local machine without exposing it, for testing (here, `localhost:8081`).

15. What is the difference between Docker and Kubernetes? Docker builds and runs containers. Kubernetes manages many containers across machines: scheduling, scaling, networking and recovery.

16. What is a rolling update? A Deployment replaces pods gradually with a new version so the app stays available. Roll back with `kubectl rollout undo`.

17. What does `kubectl apply` do differently from `kubectl create`? `apply` is declarative and can be re-run to update. `create` only makes new objects and errors if the object exists.

Experiment 5: Jenkins and Slack (CI/CD Notifications)

Theory

Continuous Integration (CI) means developers merge code into a shared repository often, and every merge triggers an automatic build and test. It finds integration bugs early. **Continuous Delivery** keeps the code always ready to release; **Continuous Deployment** releases every passing change automatically. Together they form the **CI/CD pipeline**.

Jenkins is an open-source automation server written in Java that runs these pipelines. It is driven by a web interface, runs on port 8080 by default, and gets most of its features from plugins.

Key concepts

Concept	Meaning
Job (project)	A configured task Jenkins runs, such as build and test
Freestyle job	Simple job set up through forms in the UI
Pipeline	A job defined as code in a `Jenkinsfile` (declarative or scripted)
Build	One run of a job; has a number and a result
Build result	SUCCESS, FAILURE, UNSTABLE or ABORTED
Trigger	What starts a build: manual, SCM poll, webhook, schedule

Concept	Meaning
Plugin	Add-on that extends Jenkins (Git, Maven, Slack)
Agent (node)	Machine that runs builds for the controller
Post-build action	A step that runs after the build finishes, such as notifying a team

Slack integration. Slack is a team messaging tool. Jenkins connects to it through the **Slack Notification** plugin, which posts to a channel using a token or webhook created in Slack.

1. Slack side: create a channel and add the Jenkins CI app. It gives a token or incoming webhook URL.
2. Jenkins side: install the plugin, store the token as a credential, and set the workspace and default channel under *Manage Jenkins > System*.
3. Job side: in *Post-build Actions*, choose *Slack Notifications* and select which events to report (start, success, failure, unstable, back to normal).

In a pipeline the same is written as code, for example `slackSend(channel: '#jenkins-builds', color: 'good', message: "Build ${env.BUILD_NUMBER} succeeded")` inside a `post { success { ... } failure { ... } }` block.

Why notify. The whole team sees failures within seconds, so broken builds are fixed quickly. It removes the need to watch the Jenkins dashboard.

Viva questions

1. What is Jenkins? An open-source automation server that builds, tests and deploys software, and is the common tool for CI/CD.

2. What is CI/CD? CI automatically builds and tests every code change. CD automatically delivers or deploys changes that pass.

3. What is a Jenkins job? Difference between Freestyle and Pipeline? A job is a configured task. Freestyle is configured through the UI. Pipeline is defined as code in a `Jenkinsfile` and can be version controlled.

4. What is a `Jenkinsfile`? A text file in the repository that defines the pipeline stages (build, test, deploy) as code.

5. What are plugins and which one did you use? Plugins add features to Jenkins. I used the Slack Notification plugin.

6. What is a post-build action? A step that runs after a build finishes regardless of its outcome, such as sending notifications or archiving files.

7. How do you connect Jenkins to Slack? Add the Jenkins CI app to a Slack channel to get a token, install the Slack plugin in Jenkins, save the token as a credential, set the channel, then enable Slack notifications in the job.

8. Which build events can notify Slack? Start, success, failure, unstable, aborted, not built and back to normal.

9. What are the possible build results? SUCCESS, UNSTABLE (tests failed but build compiled), FAILURE and ABORTED.

10. What is a webhook? An HTTP callback: one system sends a message to a URL when an event happens. Slack incoming webhooks let Jenkins post messages.

11. How can a build be triggered automatically? By a webhook from Git on every push, by polling the repository, or by a schedule (cron syntax).

12. What is the difference between a Jenkins controller and an agent? The controller manages jobs and the UI. Agents are machines that execute the builds.

13. How do you store secrets such as the Slack token? In the Jenkins Credentials store, not in the job script, so they stay hidden in logs.

14. How did you test the notifications? I ran a successful build, then made one fail on purpose, and checked that both messages appeared in the Slack channel.

15. Advantages of Jenkins? Free and open source, large plugin ecosystem, supports distributed builds, and integrates with Git, Maven, Docker and Kubernetes.

16. How do the five experiments fit together in one pipeline? Git holds the code and triggers Jenkins. Maven builds and tests it. Docker packages the result into an image. Kubernetes runs the image at scale. Jenkins automates the flow and Slack reports the outcome.